COSC 423/523: Introduction to Artificial Intelligence

Reinforcement Learning



THE UNIVERSITY OF TENNESSEE KNOXVILLE

Credits to UC Berkeley CS188 (Introduction to Artificial Intelligence), with thanks to Professor Pieter Abbeel for sharing the materials.

Exam 2 – Oct 30 (3:00 pm-3:50 pm)

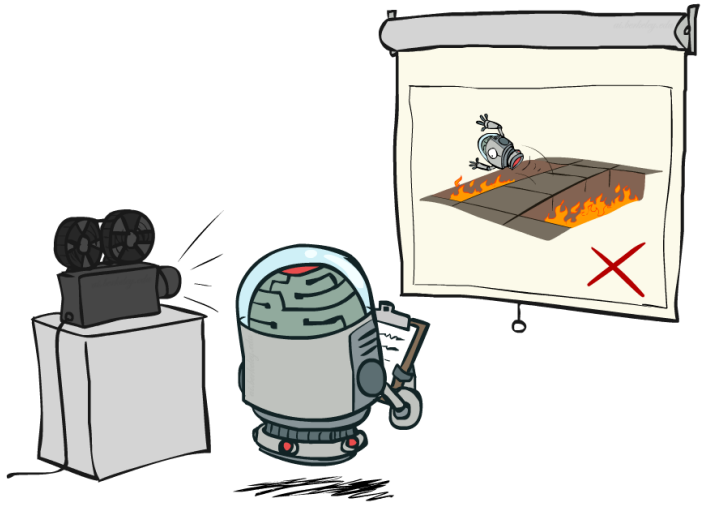
- **Topics**
  - **Lectures 16-22:** State values and Q-values; Bellman equations; Value iteration; Policy evaluation and policy iteration; Model-based and model-free learning; Monte Carlo evaluation; Temporal Difference (TD) learning; Q-learning; Approximate Q-learning

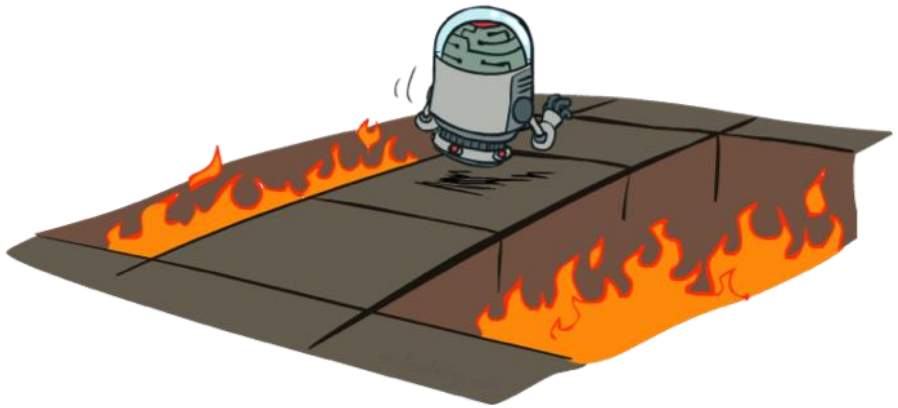
- **Format & Policies**
  - **Closed-book, closed-notes, one blank sheet of paper is allowed for scratch work only**
  - **Complete the exam on Canvas using your own laptop**
  - **20 multiple-choice questions, 1 bonus question**
    - 5 points for each multiple-choice question
    - Scores are capped at 100 points

- **Not Emphasized**
  - Long derivations or proofs; Memorizing formulas without understanding; Programming

Today

- Model-Based Learning
- Model-Free Learning
  - Passive Reinforcement Learning
    - Direct Evaluation (Monte Carlo Evaluation)
    - Temporal Difference Learning
  - Active Reinforcement Learning
    - Q-Learning





MDPs and RL

Known MDP: Offline Solution

Goal	Technique
Evaluate a fixed policy π	Policy evaluation
Compute V*, Q*, π*	Value / policy iteration

Unknown MDP: Model-Based

Goal	Technique
Evaluate a fixed policy π	PE on approx. MDP
Compute V*, Q*, π*	VI/PI on approx. MDP

Unknown MDP: Model-Free

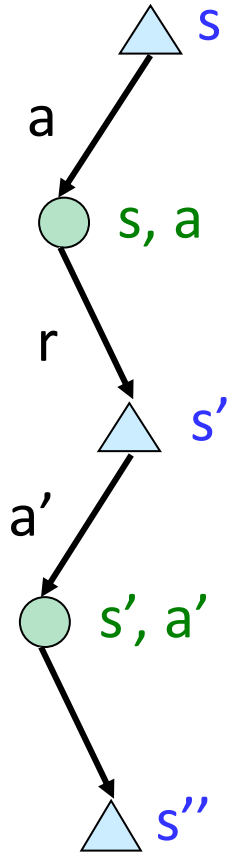
Goal	Technique
Evaluate a fixed policy π	Value Learning
Compute V*, Q*, π*	Q-learning

Model-Free Learning

- Model-free (temporal difference) learning
  - Experience world through episodes

$$(s, a, r, s', a', r', s'', a'', r'', s'''' \ldots)$$

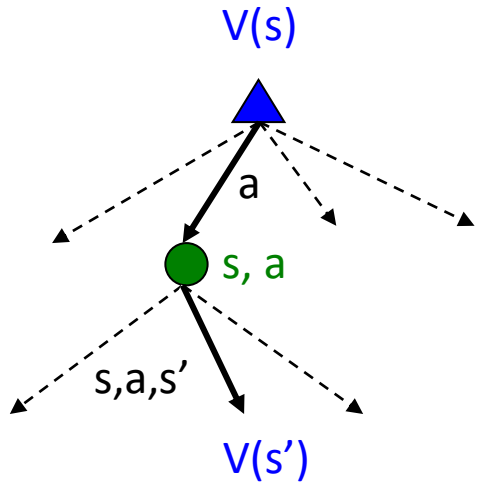
  - Update estimates each transition (s, a, r, s')
  - Over time, updates will mimic Bellman updates



Recap: Value Iteration

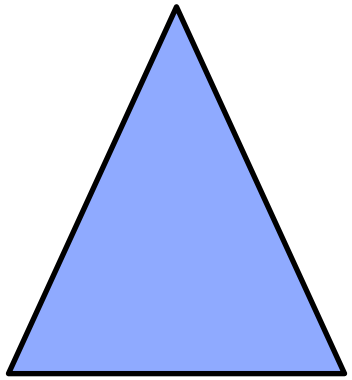
- Bellman equations characterize the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$



- Value iteration computes them:

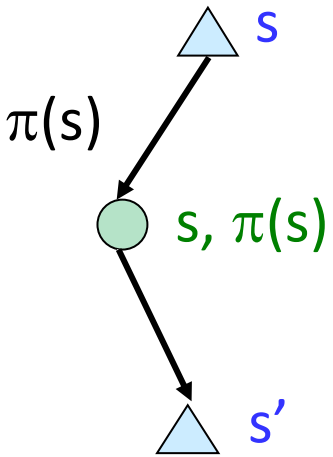
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$



- Value iteration is just a fixed point solution method
  - V_k can also be seen as the values of depth-k truncated trees
  - **Theorem.** If $0 \leq \gamma < 1$, value iteration converges to the unique optimal value function V^*

Recap: Temporal Difference Learning

- Big idea: learn from every experience!
  - Update V(s) each time we experience a transition (s, a, s', r)
  - Likely outcomes s' will contribute updates more often



- Temporal difference learning of values
  - Policy still fixed, still doing evaluation!
  - Move values toward value of whatever successor occurs: running average

Sample of V(s): $sample = R(s, \pi(s), s') + \gamma V^{\pi}(s')$

Update to V(s): $V^{\pi}(s) \leftarrow (1-\alpha)V^{\pi}(s) + (\alpha)sample$

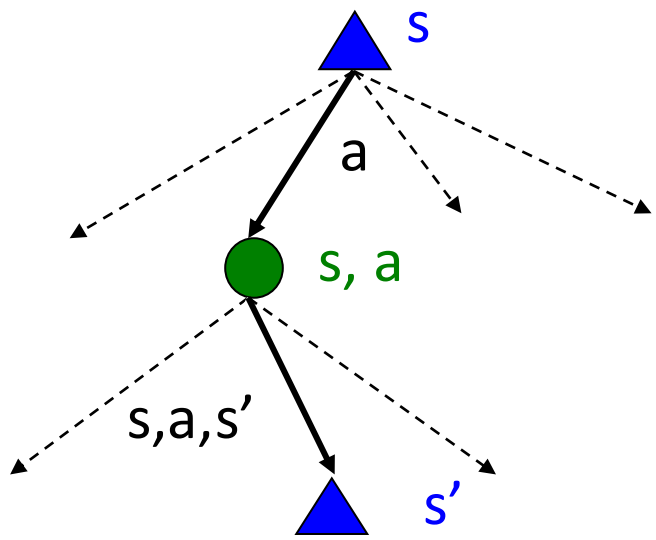
Same update: $V^{\pi}(s) \leftarrow V^{\pi}(s) + \boxed{\alpha(sample - V^{\pi}(s))}$

Problems with TD Value Learning

- TD value leaning is a model-free way to do policy evaluation, mimicking Bellman updates with running sample averages
- However, if we want to turn values into a (new) policy, we're sunk:

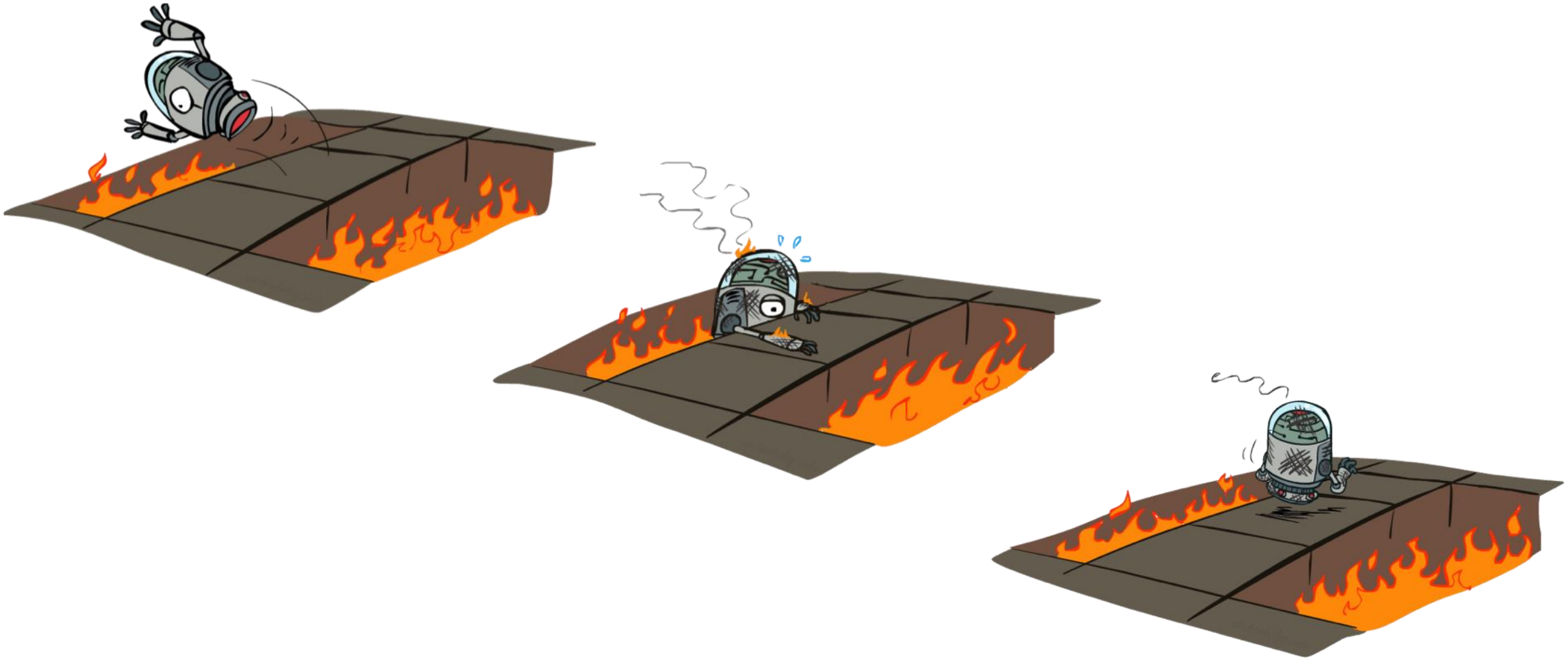
$$\pi(s) = \arg\max_a Q(s,a)$$

$$Q(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V(s') \right]$$



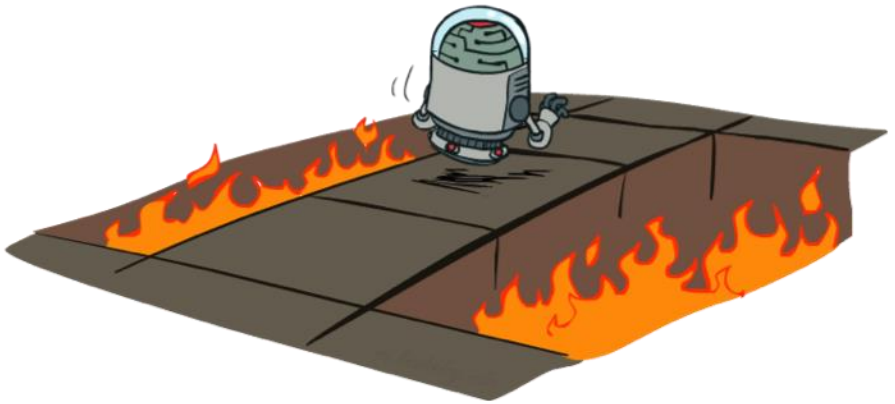
- Idea: learn Q-values, not values
- Makes action selection model-free too!

Active Reinforcement Learning



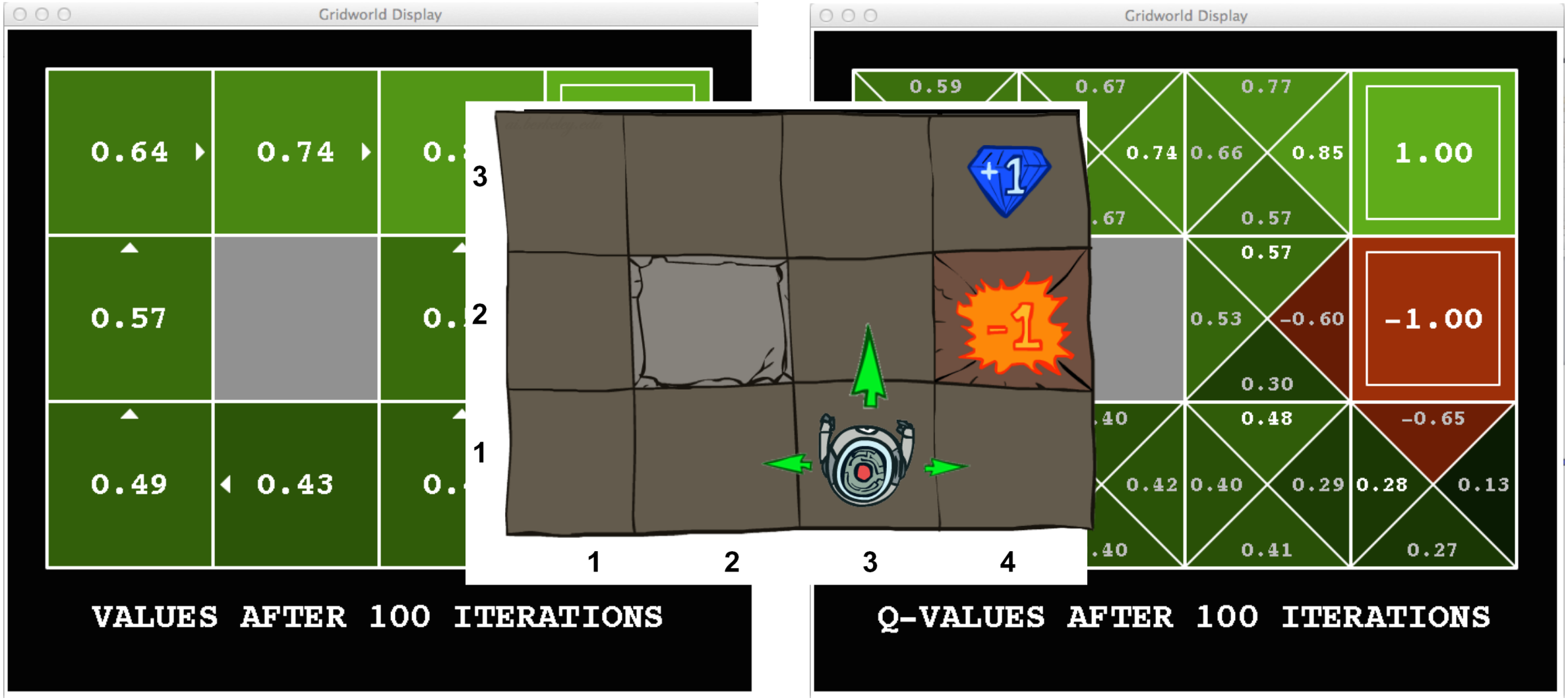
Active Reinforcement Learning

- Full reinforcement learning: optimal policies (like value iteration)
  - You don't know the transitions T(s,a,s')
  - You don't know the rewards R(s,a,s')
  - You choose the actions now
  - Goal: learn the optimal policy / values



- In this case:
  - Learner makes choices!
  - Fundamental tradeoff: exploration vs. exploitation
  - This is NOT offline planning! You actually take actions in the world and find out what happens…

Recap: Gridworld V Values



Demo: https://nowke.github.io/rlviz

Noise = 0.2, Discount = 0.9, Living reward = 0

Detour: Q-Value Iteration

- Value iteration: find successive (depth-limited) values
  - Start with $V_0(s) = 0$, which we know is right
  - Given V_k, calculate the depth k+1 values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- But Q-values are more useful, so compute them instead
  - Start with $Q_0(s,a) = 0$, which we know is right
  - Given Q_k, calculate the depth k+1 q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

Q-Learning

- Q-Learning: sample-based Q-value iteration

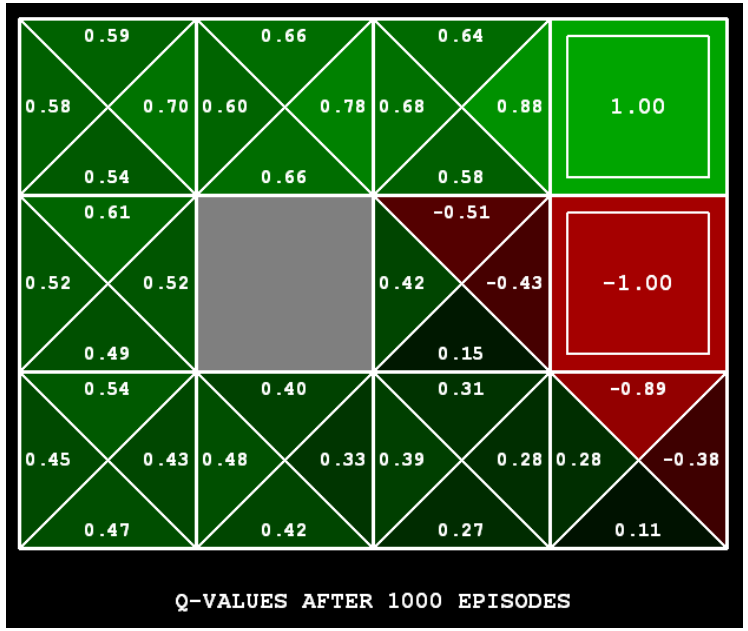
$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a') \right]$$

- Learn Q(s,a) values as you go
  - Receive a sample (s,a,r,s')
  - Consider your old estimate: $Q(s,a)$
  - Consider your new sample estimate:

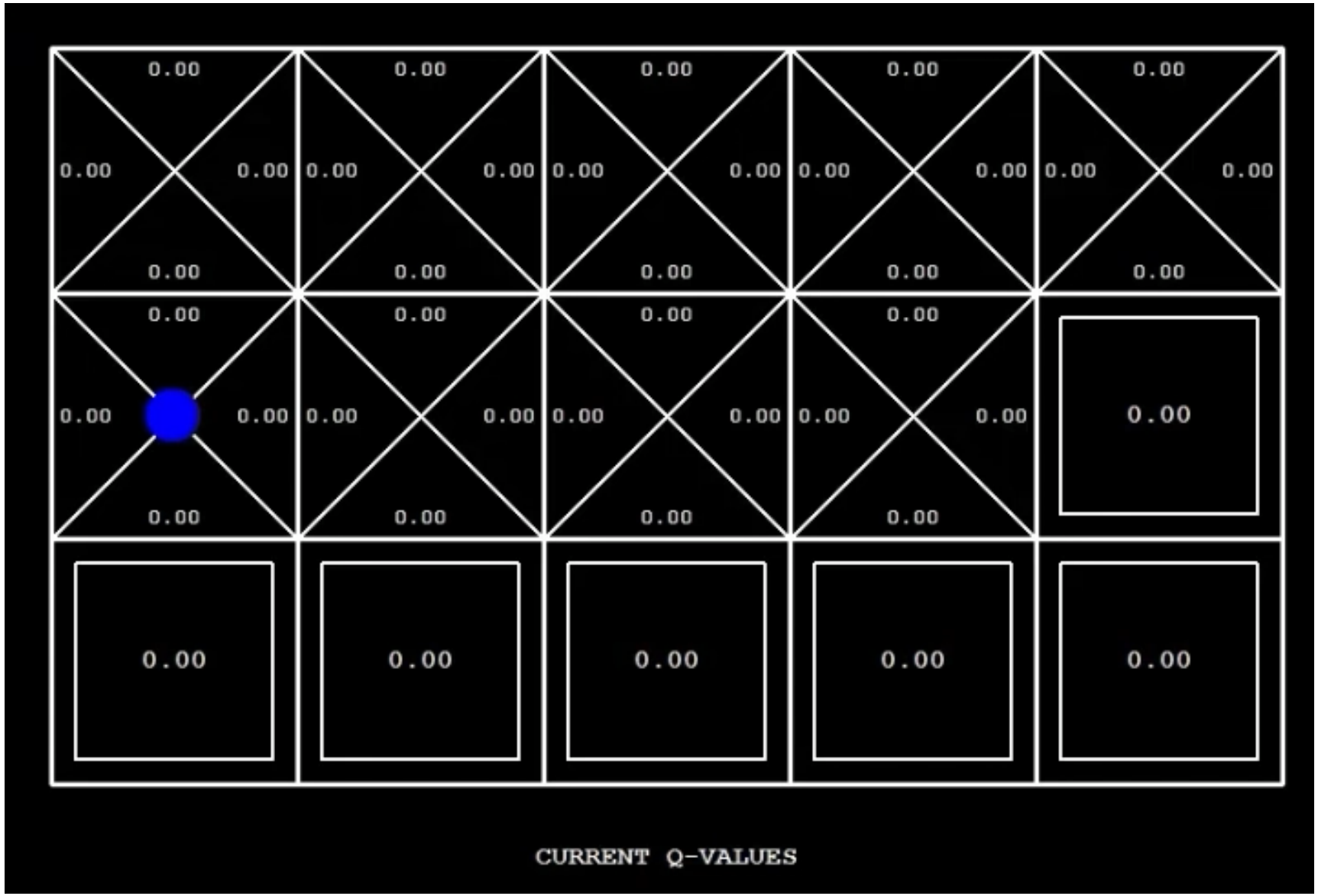
$$sample = R(s,a,s') + \gamma \max_{a'} Q(s',a')$$

  - Incorporate the new estimate into a running average:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + (\alpha)\,[sample]$$



Demo: Q-Learning



Environment Settings :

Goal on the right:
+10 reward

Falling off the cliff (bottom row):
−100 reward

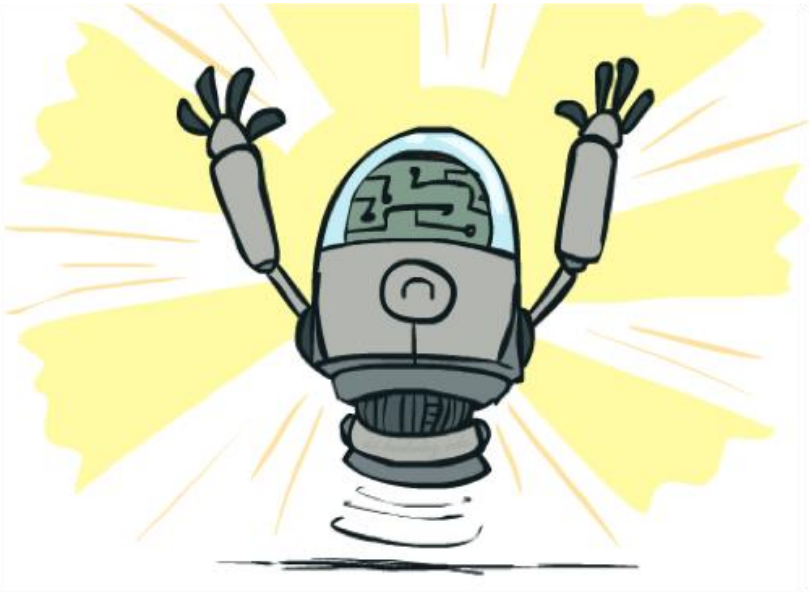
Discount factor:
γ = 1

Learning rate:
α = 0.5

Q-Learning Properties

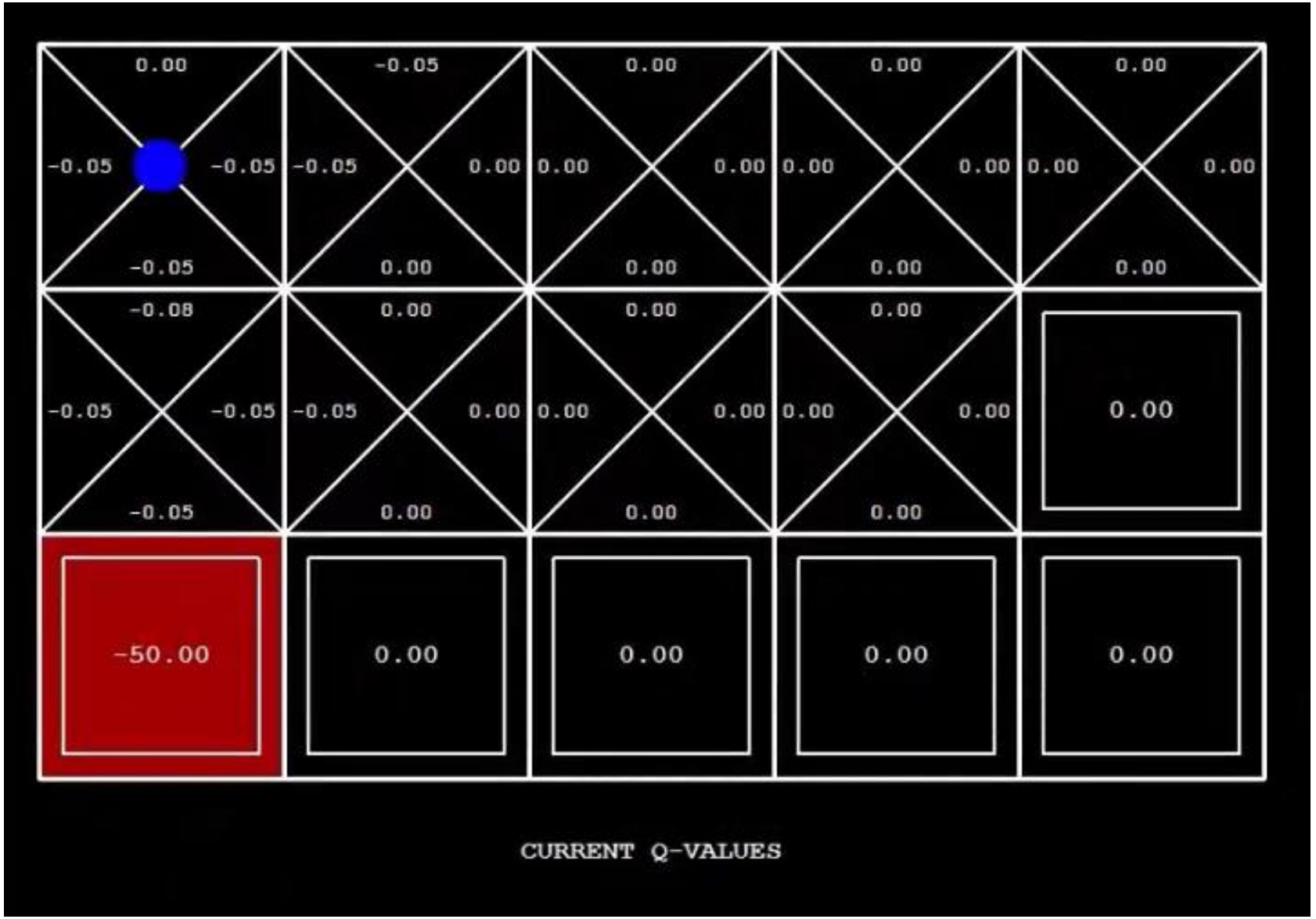
- Amazing result: Q-learning converges to optimal policy -- even if you're acting suboptimally!

- This is called off-policy learning

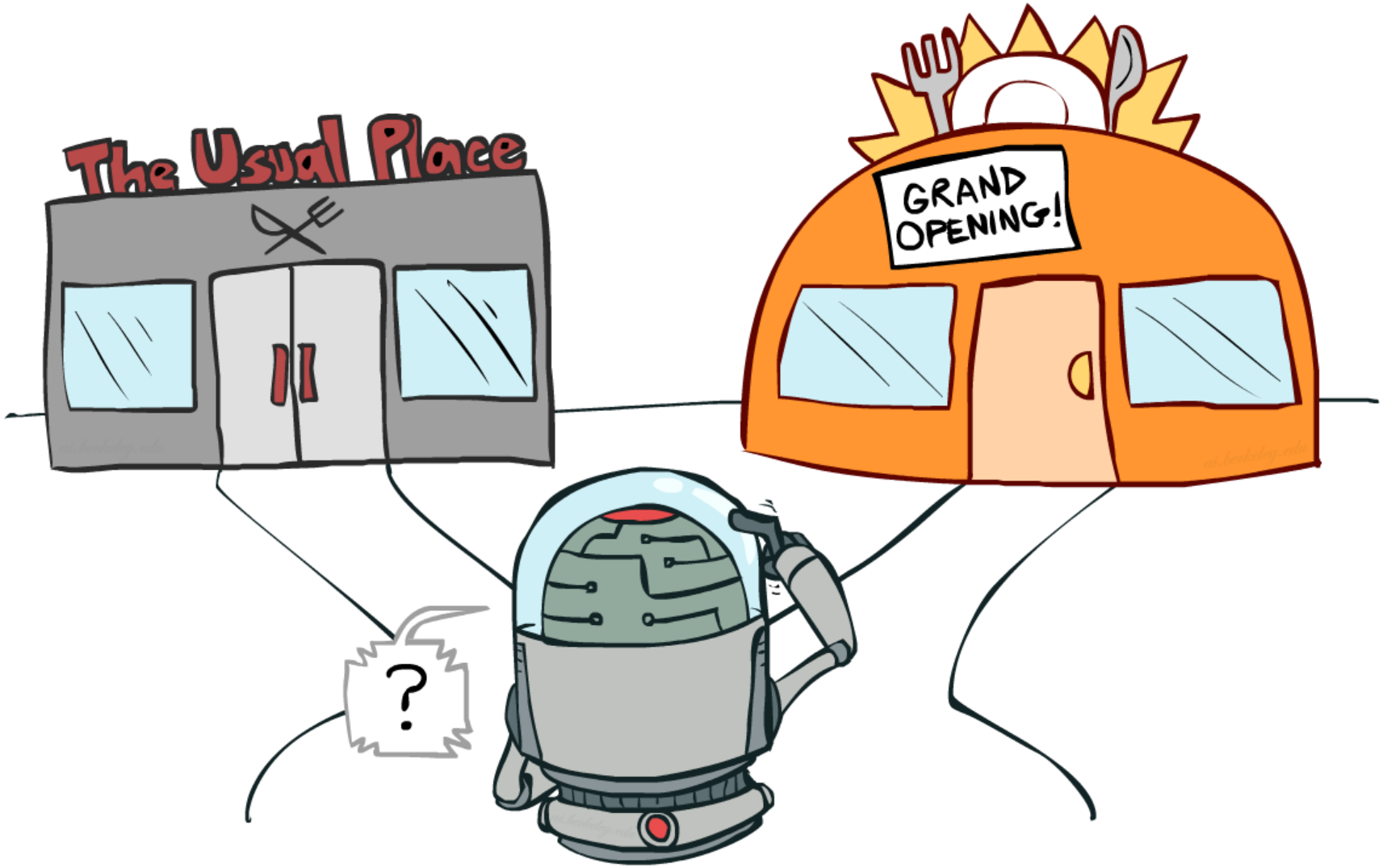


- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly
  - Basically, in the limit, it doesn't matter how you select actions (!)

Demo: Q-Learning Auto Cliff Grid



Exploration vs. Exploitation



How to Explore?

- Several schemes for forcing exploration
  - Simplest: random actions (ε-greedy)
    - Every time step, flip a coin
    - With (small) probability ε, act randomly
    - With (large) probability 1-ε, act on current policy
  - Problems with random actions?
    - You do eventually explore the space, but keep thrashing around once learning is done
    - One solution: lower ε over time
    - Another solution: exploration functions



Exploration Functions

- When to explore?
  - Random actions: explore a fixed amount
  - Better idea: explore areas whose badness is not (yet) established, eventually stop exploring



- Exploration function
  - Takes a value estimate u and a visit count n, and returns an optimistic utility, e.g. $f(u, n) = u + k/n$

Regular Q-Update: $Q(s,a) \leftarrow_\alpha [R(s,a,s') + \gamma \max_{a'} Q(s',a')]$

Modified Q-Update: $Q(s,a) \leftarrow_\alpha [R(s,a,s') + \gamma \max_{a'} f(Q(s',a'), N(s',a'))]$

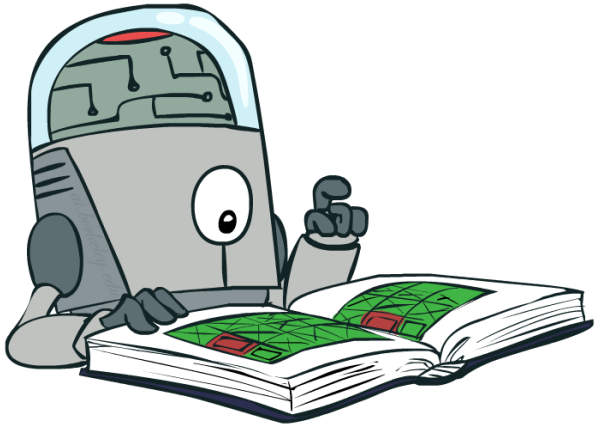
  - Note: this propagates the "bonus" back to states that lead to unknown states as well!

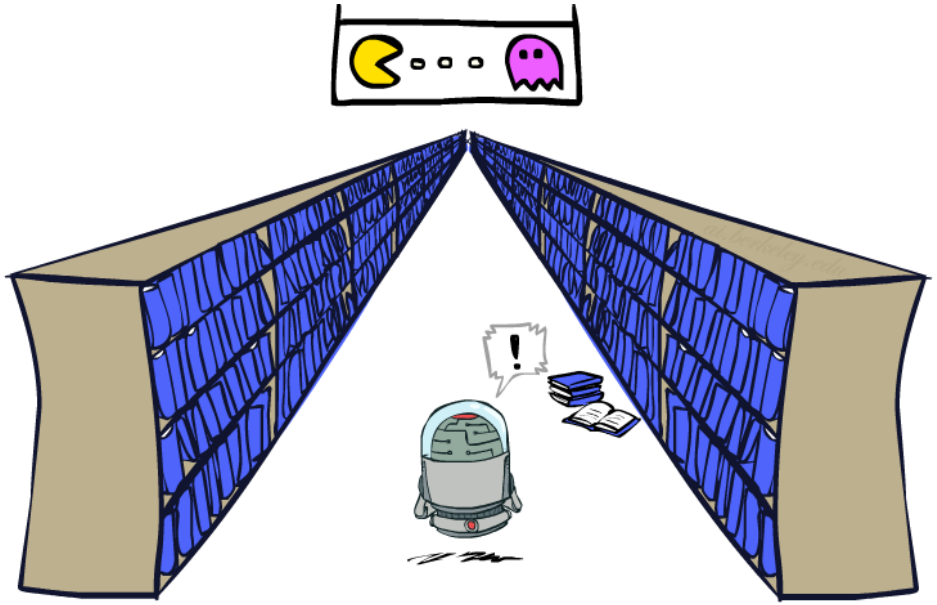
Generalizing Across States

- Basic Q-Learning keeps a table of all q-values

- In realistic situations, we cannot possibly learn about every single state!
  - Too many states to visit them all in training
  - Too many states to hold the q-tables in memory

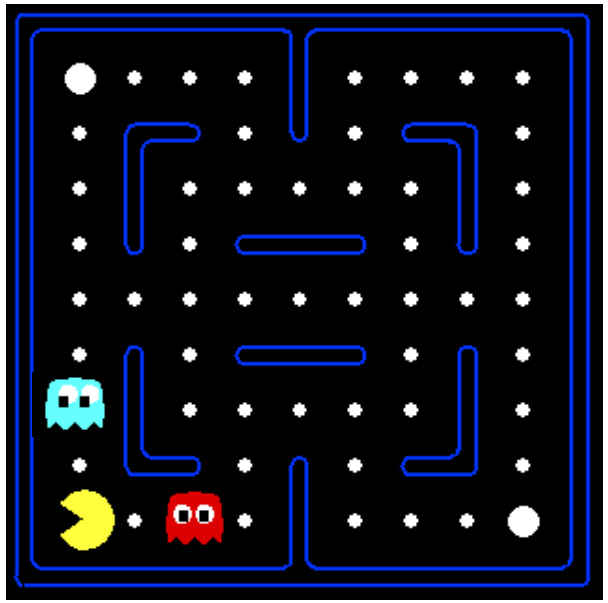
- Instead, we want to generalize:
  - Learn about some small number of training states from experience
  - Generalize that experience to new, similar situations
  - This is a fundamental idea in machine learning, and we'll see it over and over again



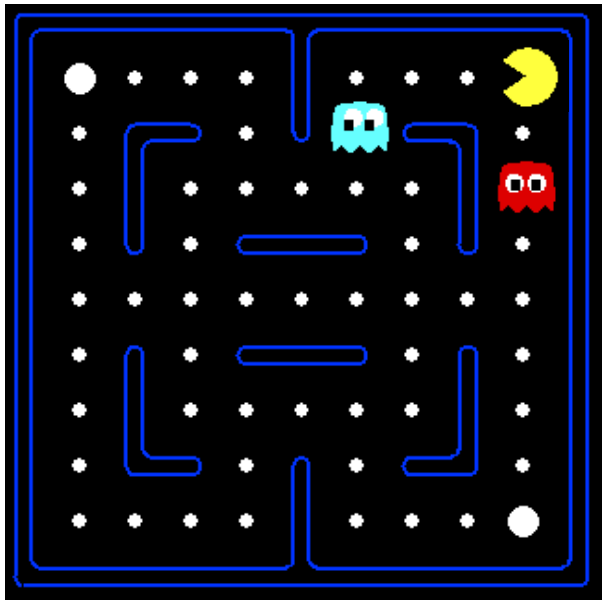


Example: Pacman

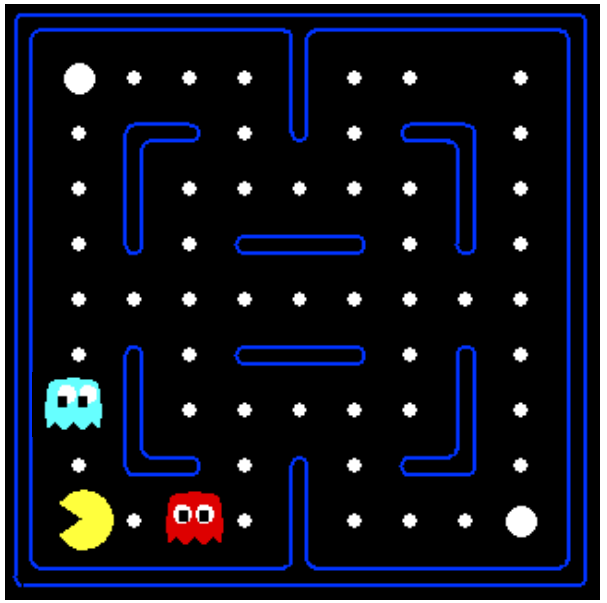
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:



Or even this one!



Next Lecture: Approximate Q-Learning

